How Do LLM-Based Agents Represent Code Repositories? A Systematic Literature Review

ANONYMOUS AUTHOR(S)

Large language models (LLMs) increasingly serve as the backbone of automated software engineering agents tasked with completing code, resolving issues, and generating tests across entire repositories. The quality of these agents is fundamentally bounded by how they *represent* the repository as context. This paper presents a systematic literature review (SLR) of 163 primary studies (2020–2026), proposing a taxonomy of seven *Paradigm of Automated Repository Representation* (PAR) categories—dense embeddings, cross-file RAG, graph-based structures, execution traces, memory augmentation, compression, and agentic navigation—characterised on ten structural dimensions. As our central contribution, we develop a formal trade-off framework that evaluates each paradigm on five operational dimensions: Semantic Precision, Token Efficiency, Construction Latency, Update Cost, and Scalability. The framework produces a comparative matrix and five practitioner decision guidelines, supported by evidence from 29 focal studies. We find that no paradigm dominates all dimensions, identify three compositional hybrid strategies that approach the Pareto frontier, and articulate five open research problems.

Additional Key Words and Phrases: repository-level code generation, LLM-based software agents, code context representation, retrieval-augmented generation, systematic literature review, trade-off framework

ACM Reference Format:
Anonymous Author(s). 2026. How Do LLM-Based Agents Represent Code Repositories? A Systematic Literature Review. *ACM Trans. Softw. Eng. Methodol.* 0, 0, Article 0 (2026), 14 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

The ability of large language models (LLMs) to reason about entire software repositories—not merely individual functions or files—has emerged as a cornerstone challenge in AI-assisted software engineering. Industrial deployment of LLM-based coding agents, from GitHub Copilot Workspace to SWE-agent [35] and OpenHands [31], requires these systems to *represent* repositories in a form that fits within finite context windows while preserving the semantic information needed to resolve issues, generate features, and refactor code.

The central question we address in this survey is: *How do LLM-based agents represent code repositories?* Answering this question matters because the representation paradigm chosen by a system fundamentally determines its capability ceiling: systems that flatten repositories into dense embedding vectors (PAR-1) achieve sub-millisecond retrieval but discard relational structure; systems that build call graphs (PAR-3) expose structural dependencies but incur quadratic indexing cost on large codebases; agentic navigators (PAR-7) adapt their context dynamically but multiply latency by the number of tool calls. No single paradigm dominates all operational dimensions.

We conduct a systematic literature review (SLR) following Kitchenham and Charters guidelines [16], augmented with forward/backward snowballing [33], yielding a corpus of **163** primary studies published between 2020 and early 2026. Our contributions are fourfold:

(1) A seven-category taxonomy of repository representation paradigms (PAR-1 through PAR-7), characterised on ten structural dimensions (§3).

2026. ACM 1557-7392/2026/0-ART0
https://doi.org/10.1145/nnnnnnn.nnnnnnn

(2) An evidence-based comparison of benchmark performance across paradigms (§4).
(3) A *formal trade-off framework* that operationalises the choice between paradigms on five engineering-critical dimensions (§5).
(4) An agenda of open research problems and design guidelines for practitioners (§6).

The remainder of this paper is organised as follows. Section 2 describes our SLR methodology. Section 3 presents the taxonomy. Section 4 reports empirical findings. Section 5 develops the trade-off framework. Section 6 discusses implications and limitations. Section 7 surveys related work. Section 8 concludes.

2 Methodology

We followed a systematic literature review (SLR) protocol based on the guidelines of Kitchenham and Charters [16], complemented by the snowballing procedure of Wohlin [33] and PRISMA 2020 reporting standards [25].

2.1 Research Questions

RQ1 What paradigms do LLM-based agents use to represent code repositories?
RQ2 How do these paradigms perform empirically across benchmark tasks (code completion, issue resolution, test generation)?
RQ3 What are the operational trade-offs among paradigms, and how should practitioners select among them?

2.2 Search Strategy

We queried ACM Digital Library, IEEE Xplore, arXiv, and Semantic Scholar using a Boolean query combining terms from three facets: (i) *repository-level* context or codebase understanding; (ii) *LLM / language model* or neural code model; (iii) *retrieval / representation / agent*. The search covered publications from January 2020 to February 2026.

2.3 Inclusion and Exclusion Criteria

We included peer-reviewed conference papers, journal articles, and technical reports that: (a) propose or evaluate a method that represents a multi-file code repository as input to an LLM, and (b) report at least one quantitative result on a code-related task. We excluded short abstracts (<4 pages), tutorials, and studies restricted to single-file or single-function contexts.

2.4 Study Selection

Initial search returned 1,247 candidates. After title/abstract screening, 412 full-text papers were reviewed. Forward and backward snowballing added 38 papers. Application of inclusion/exclusion criteria yielded a final corpus of **163** primary studies.

2.5 Data Extraction

Two independent reviewers extracted the following fields: representation paradigm, LLM backbone, benchmark datasets, performance metrics, token budget, and deployment setting. Inter-rater agreement was $\kappa = 0.81$ (substantial).

2.6 Synthesis

We applied thematic synthesis [4] to derive the seven PAR categories and ten characterisation dimensions. Quantitative results were aggregated via weighted vote-counting where sufficient primary data were available.

2.7 Validity Threats

Publication bias may over-represent successful paradigms. Rapid arXiv publishing inflates PAR-7 counts due to the agentic trend. Metric heterogeneity across benchmarks limits direct comparisons. We mitigated these threats by including grey literature and normalising metrics where possible.

3 Taxonomy of Repository Representation Paradigms

We identify seven *Paradigm of Automated Repository Representation* (PAR) categories, differentiated along ten structural dimensions: (D1) granularity of retrieval unit, (D2) index structure, (D3) retrieval mechanism, (D4) context assembly strategy, (D5) cross-file dependency modelling, (D6) dynamic context update, (D7) multi-modal artefact support, (D8) downstream task scope, (D9) evaluation benchmark, and (D10) training dependency (Zero-shot L0, RAG-only L1, Lightweight FT L2, Full RL L3).

3.1 PAR-1: Dense Embedding Retrieval

PAR-1 systems encode repository artefacts as fixed-size dense vectors and retrieve relevant snippets via approximate nearest-neighbour search. Representative systems include CodeBERT [6], GraphCodeBERT [10], and UniXcoder [9].

3.2 PAR-2: Retrieval-Augmented Generation (Cross-File)

PAR-2 systems perform query-driven retrieval from a file-level index and prepend retrieved chunks to the LLM prompt. RepoCoder [36], Repoformer [34], ReACC [21], and RepoGenix [17] exemplify this paradigm.

3.3 PAR-3: Graph-Based Structural Representation

PAR-3 systems build explicit structural graphs: Abstract Syntax Trees (ASTs), Control Flow Graphs (CFGs), Call Graphs, Data Flow Graphs (DFGs), or Knowledge Graphs. GraphCoder [18], RepoGraph [24], RepoHyper [27], and CodexGraph [20] belong to this paradigm. GraphCodeBERT [10] integrates a DFG during pre-training and straddles PAR-1/PAR-3.

3.4 PAR-4: Execution-Trace and Dynamic Context

PAR-4 systems derive context from runtime artefacts (stack traces, test output, compiler errors) or passive static-analysis traces rather than constructing an explicit offline index. SWE-agent [35] and AutoCodeRover [39] exemplify reactive, execution-driven context assembly.

3.5 PAR-5: Memory-Augmented Representation

PAR-5 systems maintain a persistent, mutable memory store that *is* the context window for a given episode. CodeMEM [30] uses AST-guided adaptive memory; SWE-Exp [2] accumulates experience traces across issues. Unlike PAR-7 (where memory is one component of a broader agent loop), in PAR-5 the memory store is the primary representation substrate.

3.6 PAR-6: Compression and Summarisation

PAR-6 systems reduce token footprint by summarising or compressing repository content before passing it to the LLM. LongCodeZip [28] learns differentiable soft-token compression; hierarchical pruning approaches [5, 38] and SWE-Pruner [32] apply hierarchical or agent-driven summarisation.

3.7 PAR-7: Agentic Repository Navigation

PAR-7 systems employ an autonomous agent loop—tool calls, file reads, searches, code execution—to *navigate* the repository on demand. Memory is one component among many. CodePlan [1], MetaGPT [11], OpenHands [31], LingmaAgent [23], and Prometheus [26] belong to this paradigm.

4 Empirical Findings

This section answers RQ1 (paradigm distribution) and RQ2 (benchmark performance). A detailed per-study breakdown is available in our online replication package.

4.1 Paradigm Distribution

Of the 163 primary studies, PAR-2 (RAG-based retrieval) accounts for the largest share (38%), reflecting the accessibility of retrieval pipelines. PAR-7 (agentic) grew from 2% of publications in 2021 to 29% in 2025, mirroring the SWE-bench [15] evaluation trend. PAR-3 (graph-based) constitutes 18% and PAR-6 (compression) 9%. PAR-1, PAR-4, and PAR-5 account for the remaining 6%.

4.2 Benchmark Performance

On RepoBench [19], PAR-3 systems achieve the highest exact-match scores (avg. 52.4%) by exploiting structural dependency edges; PAR-2 systems score 47.1% on average. On SWE-bench [15], PAR-7 agentic systems dominate with a resolution rate of 35–50% (*e.g.* LingmaAgent [23]), compared with 12–18% for non-agentic PAR-2 pipelines. HumanEvo [40] reveals temporal data leakage: PAR-2 systems evaluated on evolved (post-training-cutoff) repositories degrade by up to 14% relative to static benchmarks, exposing freshness bias.

4.3 Discussion of RQ1 & RQ2

The empirical picture reveals a consistent pattern: structural fidelity (PAR-3) pays off on completion tasks with stable APIs, while adaptability (PAR-7) dominates issue resolution where the agent must *discover* the relevant code locus. Neither paradigm is universally superior; the choice depends on operational context. We formalise this intuition in the next section.

5 A Formal Trade-off Framework for Representation Selection

"The choice of context representation is not an implementation detail—it is the primary design decision that caps a system's quality ceiling." — derived from the SWE-bench analysis of Yang et al. [35]

Sections 3 and 4 established *what* the seven PAR paradigms do and *how well* they perform in controlled benchmarks. Section 5 addresses the practitioner's question: *given a deployment context, which paradigm should I choose?* We answer this by constructing a **formal trade-off framework** grounded in five operationally measurable dimensions, deriving a

scored comparison matrix, and stating explicitly when and why a practitioner should prefer PAR-X over PAR-Y.

5.1 Motivation: Why One Dimension Is Never Enough

Prior surveys present performance tables organised by benchmark score [12, 13, 29]. While informative, this single-axis ranking conflates fundamentally different cost categories. Consider two contrasting failure modes:

- **Repoformer** [34] achieves high token efficiency by selectively skipping retrieval for in-file-sufficient queries, but its static retrieval policy cannot adapt when a freshly pushed commit invalidates the indexed embedding space.
- **SWE-agent** [35] dynamically discovers context by navigating the live repository, achieving near-perfect freshness, but issues 30–60 tool calls per task, incurring latency of 2–10 minutes on SWE-bench instances.

These failure modes are orthogonal: a system can be fast yet stale, or fresh yet slow. A practitioner deploying a CI/CD copilot (*e.g.* sub-second inline completion) faces entirely different constraints than one deploying a nightly issue-resolution agent (*e.g.* SWE-bench-style). The framework we introduce makes these constraints explicit.

5.2 The Five Operational Dimensions

We define five operational dimensions $\mathcal{D}_1$–$\mathcal{D}_5$ that jointly determine the cost–quality surface of a repository representation paradigm. Each dimension is assigned a score from 1 (worst) to 5 (best) based on evidence from primary studies.

$\mathcal{D}_1$: Semantic Precision. Definition: The fraction of task-relevant context that the paradigm includes in the assembled prompt, excluding irrelevant noise. Formally, let R be the set of code elements causally required to solve a task, and let A be the set assembled by the paradigm. Semantic precision is $|R \cap A|/|A|$ (precision) and $|R \cap A|/|R|$ (recall); the harmonic mean F_1 serves as our proxy.

Evidence: Graph-based systems (PAR-3) achieve the highest F_1 on cross-file completion because dependency edges encode precise causal links [3, 18]. GraphCoder's coarse-to-fine retrieval over code-context graphs raises cross-file F_1 by +7.3 pp over BM25-based RAG [18]. PAR-2 systems with BM25 or dense retrieval suffer precision degradation at scale: Gu et al. [7] show that *what* is retrieved matters more than *how much*, and that token-budget saturation (¿ 80% of context filled with irrelevant chunks) cuts downstream performance by 18%. PAR-7 agentic systems achieve high recall by design (they can issue unlimited tool calls) but at the cost of precision: the agent may navigate several red-herring files before finding the root cause [23, 35]. PAR-6 compression introduces precision loss when the summariser discards semantically thin but syntactically critical tokens (*e.g.* type annotations, exception signatures), a deficiency documented by LongCodeZip [28] and quantified by SWE-Pruner [32]: over-aggressive pruning incurs a -11% task-completion regression on SWE-bench.

$\mathcal{D}_2$: Token Efficiency. Definition: The ratio of semantically informative tokens to total tokens consumed, averaged over a task batch. A higher ratio implies more information per dollar of LLM API cost.

Evidence: PAR-6 compression maximises token efficiency almost by definition: LongCodeZip [28] achieves a 4× compression ratio (i.e., 1,000 effective tokens from a 4,000-token context) with <3% downstream quality loss on code completion benchmarks. Repoformer [34] selectively skips retrieval on 35–55% of queries that do not benefit from cross-file context, saving one retrieval call and associated tokens per skipped query with no quality regression. PAR-3 graph systems incur a token overhead because graph-serialisation formats (adjacency lists, Cypher queries [20]) are verbose; CodexGraph's Cypher interface reduces raw token usage vs. full-graph serialisation but still costs 20–40% more tokens than a pure BM25 RAG pipeline for the same recall level. PAR-7 agentic systems are worst: each tool call consumes a full model forward pass plus the tool's output, and multi-step navigation inflates total tokens by 3–8× relative to a single-shot PAR-2 retrieval [31]. HumanEvo [40] further shows that token-budget waste in PAR-7 systems grows super-linearly with repository size, because agents explore an increasing number of irrelevant file paths.

$\mathcal{D}_3$*: Construction Latency. Definition*: Wall-clock time to produce the context that is submitted to the LLM, measured in a *cold-start* scenario (first query on a previously unseen repository).

Evidence: PAR-1/PAR-2 systems with offline-indexed embeddings achieve <100 ms retrieval latency after indexing, making them suitable for interactive (sub-200 ms) code completion. Repoformer [34] adds a negligible classification overhead (<5 ms) to decide whether retrieval is needed at all. PAR-3 systems incur indexing latency proportional to repository size: building a project-level call graph for a 50K-LOC Java repository takes 8–45 seconds depending on the tool chain [18, 24]. At query time, graph traversal is fast (<50 ms) for bounded-hop queries; however, whole-graph operations for CodexGraph-style Cypher queries [20] can reach 300–800 ms. PAR-4 execution-trace systems depend on runtime: triggering a test suite to obtain a stack trace takes 10s of seconds to minutes. PAR-6 compression adds an amortisable one-time cost to summarise repository files (∼30–120 s for a 10K-LOC codebase [28]), but the compressed representation then serves any subsequent query instantly. PAR-7 agentic navigation is the highest-latency paradigm: median first-response time on SWE-bench is 180–450 seconds for competitive agents [23, 35].

$\mathcal{D}_4$*: Update Cost. Definition*: Incremental time and compute required to keep the representation consistent after a single commit modifies a subset of the repository.

Evidence: PAR-2 RAG systems with chunk-level indices support cheap *incremental* updates: only modified file chunks need re-embedding. Repoformer [34] tracks file modification timestamps and re-indexes only changed chunks, keeping update latency $O(\Delta)$ (proportional to diff size). PAR-3 graph systems face a more expensive update: a commit that renames a public API function invalidates all call-graph edges to that function, potentially requiring whole-graph re-computation for languages without incremental static-analysis tooling [18, 24]. HumanEvo [40] stress-tests this by simulating repository evolution across 12 months of real commit history; PAR-3 systems degrade 14–22% more than PAR-2 systems in the absence of incremental graph updates. CodeMEM [30] mitigates update cost by representing repository state as a mutable in-context memory tree that patches locally (AST node replacement) rather than globally; update cost is O(diff lines). PAR-7 agentic systems have the lowest update cost: because they hold no persistent index, they always read the live filesystem and are inherently up-to-date, albeit at the cost of re-navigating from scratch on every invocation [35]. PAR-6 compression systems must re-summarise changed files; intelligent change-propagation strategies are largely absent from the current literature (a gap noted in §5.6).

Table 1. Trade-off Matrix: PAR Paradigms on Five Operational Dimensions. Scores 1–5 (5 = best). **Bold** indicates the highest score per column. ↓ indicates lower is better for $\mathcal{D}_3$ and $\mathcal{D}_4$ (we invert for display so 5 = lowest latency / lowest update cost).

Paradigm	$\mathcal{D}_1$ *Semantic Precision*	$\mathcal{D}_2$ *Token Efficiency*	$\mathcal{D}_3$ *Construct. Latency*$^{\downarrow}$	$\mathcal{D}_4$ *Update Cost*$^{\downarrow}$	$\mathcal{D}_5$ *Scalability*
PAR-1: Dense Embedding	3	3	**5**	4	3
PAR-2: RAG Cross-File	3	4	**5**	**5**	4
PAR-3: Graph-Based	**5**	2	3	2	3
PAR-4: Execution-Trace	4	3	2	**5**	3
PAR-5: Memory-Augmented	4	3	4	4	3
PAR-6: Compression	2	**5**	4	3	**5**
PAR-7: Agentic Navigation	4	1	1	**5**	2

Evidence base: $\mathcal{D}_1$: [7, 18, 32]; $\mathcal{D}_2$: [20, 28, 31, 34]; $\mathcal{D}_3$: [18, 23, 28, 34, 35]; $\mathcal{D}_4$: [30, 34, 35, 40]; $\mathcal{D}_5$: [8, 22, 26–28].

$\mathcal{D}_5$: Scalability. Definition: Degradation in $\mathcal{D}_1$–$\mathcal{D}_4$ as repository size grows from 1K to 10K+ files.

Evidence: PAR-1 dense-embedding indices scale *sub-linearly* in retrieval time thanks to ANNS (approximate nearest-neighbour search) data structures, but their semantic coverage degrades at scale because a fixed embedding dimension cannot distinguish functionally similar modules [8]. SECRET [8] addresses this with segmented deep hashing and demonstrates +8% retrieval quality at the 10K-file scale vs. standard FAISS indexing. PAR-3 graph systems face $O(N^2)$ edge growth for densely interconnected repositories; RepoHyper [27] prunes the hypergraph to k-nearest structural neighbours, maintaining $O(N \log N)$ index size. PAR-7 agentic systems show the starkest scalability challenge: Prometheus [26] documents that agent trajectory length grows linearly with codebase size, hitting context-window limits (∼200K tokens for GPT-4o) on repositories exceeding ∼15K files. Luo et al.'s RPG-Encoder [22] proposes universal repository embeddings trained to be robust across scales, reporting stable performance from 500 to 50K files. PAR-6 compression systems exhibit the best scalability profile because the compressed representation is bounded by a fixed output token count regardless of repository size [28].

5.3 Trade-off Matrix

Table 1 summarises the five-dimensional scoring for each PAR paradigm. Scores range from 1 (poor) to 5 (excellent) and are assigned based on the evidence reviewed in §5.2. An ⊕ marker indicates a hybrid system that transcends its primary category.

Several patterns emerge from Table 1:

- **No paradigm dominates.** Each paradigm leads on at most two dimensions and trails on at least one. The closest to an all-rounder is PAR-2, which scores ≥ 3 on all dimensions—but it never leads on precision or scalability.
- **Precision and token efficiency are anti-correlated.** PAR-3 achieves the highest semantic precision (5) at the cost of the second-lowest token efficiency (2). PAR-6 achieves the highest token efficiency (5) at the cost of the lowest precision (2). The fundamental tension is that rich structural information is verbose, and compression discards structural nuance.
- **Latency and update cost are positively correlated with precision for structural paradigms.** PAR-3's high precision comes from expensive offline indexing (latency 3) and fragile incremental update (cost 2). PAR-7's on-demand navigation achieves high freshness (update cost 5) at the price of catastrophic construction latency (1).
- **Scalability is an Achilles' heel for agentic systems.** PAR-7 scores only 2 on scalability, the joint worst alongside PAR-3 (when graph size is unbounded). This is a critical constraint for industrial deployment.

5.4 Practitioner Selection Guidelines

We now operationalise the matrix into concrete *decision rules* for system designers. We frame each guideline as a conditional statement on the deployment context.

5.4.1 Guideline G1: Choose PAR-2 when latency and update freshness are hard constraints. Interactive tools such as IDE autocompletion have a perceptual latency budget of $\leq$200 ms [34]. PAR-2 with offline chunk-level indexing is the only paradigm that consistently satisfies this budget after the cold-start indexing phase. Its incremental update model (re-embed only changed chunks) sustains freshness across rapid commit cycles. Repoformer's selective-retrieval optimisation [34] further reduces token waste for in-file-sufficient queries. **Practical implication**: a production code-completion copilot should default to PAR-2; the selective-retrieval gate of Repoformer should be treated as a mandatory engineering best practice rather than an optional add-on.

Why not PAR-1? PAR-1 shares PAR-2's latency advantage but lacks query-conditioned retrieval; it assembles context based on the query embedding, missing cross-file dependencies that lie in semantically distant but structurally adjacent files. GraphCodeBERT [10] partially bridges this gap via DFG pre-training, but it remains a retrieval encoder, not a context assembler. **Verdict**: prefer PAR-2 over PAR-1 for any completion task requiring cross-file context.

5.4.2 Guideline G2: Choose PAR-3 when downstream task quality is the primary objective and repository churn is low. When token budget is generous, commit rate is low (*e.g.* a released library rather than an active development branch), and task quality is paramount (*e.g.* a production code-review assistant), PAR-3 graph-based representation offers the highest semantic precision. GraphCoder [18]'s coarse-to-fine retrieval over code-context graphs consistently outperforms PAR-2 baselines by 5–9 pp on RepoBench's cross-file completion categories. RepoHyper [27]'s graph-pruning strategy limits memory growth on large repositories, making PAR-3 feasible at moderate scale.

Why not PAR-3 for high-churn repositories? HumanEvo [40] quantifies the staleness penalty: a PAR-3 system evaluated on 6-month-evolved repositories (without graph refresh)

degrades by 14–22% relative to a contemporaneously evaluated PAR-2 system. Since call-graph refresh after a significant API refactor is a non-trivial engineering operation, PAR-3 becomes risky in rapid-release contexts. **Verdict**: prefer PAR-3 for static or slow-moving codebases; fall back to PAR-2 otherwise.

5.4.3 Guideline G3: Choose PAR-6 when operating under extreme token budget constraints or serving very large repositories as a single prompt. Large-context LLMs (128K–1M token windows) create a temptation to simply concatenate all repository files. In practice, input-length degradation ("lost in the middle" effects) and inference cost make raw concatenation infeasible above ∼10K files. PAR-6 compression provides a principled solution: LongCodeZip [28] achieves a 4× compression factor with <3% quality loss, enabling a 10K-LOC repository to fit in a 32K-token window. Hierarchical summarisation [5] further adapts compression depth by directory tier.

Why not PAR-6 for tasks requiring structural precision? LongCodeZip's training objective minimises cross-entropy on code tokens, which biases compression toward *frequent* patterns. Rare but critical patterns—custom exception hierarchies, obscure type constraints, deprecated-but-still-called APIs—are disproportionately likely to be elided [32]. SWE-Pruner documents a −11% regression on SWE-bench when aggressive compression removes issue-relevant type annotations [32]. **Verdict**: use PAR-6 when summarisation-robust tasks (documentation generation, high-level planning) dominate; combine PAR-2 with PAR-6 for a hybrid that retrieves precisely then compresses aggressively.

5.4.4 Guideline G4: Choose PAR-7 when the task requires open-ended exploration and latency is not a constraint. SWE-bench [15] represents the class of tasks where neither the relevant file set nor the root-cause code locus is known in advance. PAR-7 agentic systems are the only paradigm that systematically outperforms all others on this task class, achieving 35–50% resolution rates versus <18% for PAR-2 [23, 35]. The key insight is that *exploration is semantically necessary*: the agent must determine which file to read before a meaningful context can be assembled. No offline index can replicate this capability for arbitrary, unseen issues.

Why not PAR-7 for large repositories? Prometheus [26] identifies a *horizon collapse* failure mode: agentic trajectories for issues in repositories with >15K files frequently exhaust the LLM's context window on navigation scaffolding before reaching the root-cause file. LingmaAgent [23] mitigates this via a two-stage coarse-to-fine localization; however, even this system degrades by 18% on the largest SWE-bench repositories. **Verdict**: use PAR-7 unconditionally for complex issue-resolution tasks on repositories up to ∼10K files; deploy PAR-7 with a coarse-localisation pre-filter (PAR-3 or PAR-2) for larger repositories.

5.4.5 Guideline G5: Prefer PAR-5 for iterative tasks where context must accumulate across multiple LLM invocations. Tasks such as incremental feature development [30], multi-turn refactoring, or long-horizon test generation require the representation to *grow* as the agent produces and observes code changes. PAR-5 memory systems maintain a mutable, task-scoped representation that absorbs each edit delta without discarding prior context. CodeMEM [30] demonstrates a +9.1% improvement over PAR-2 on multi-turn code generation by preserving AST-node-level change history in memory. **Verdict**: for single-turn tasks PAR-5 offers no advantage over PAR-2; activate PAR-5 only when inter-turn dependency is structurally necessary.

5.5 Hybrid and Compositional Strategies

Real-world deployments increasingly combine paradigms rather than applying a single one. We identify three architecturally motivated hybrid strategies:

H1: PAR-2 + PAR-3 (precision-guided retrieval). Use a lightweight BM25/dense retriever (PAR-2) as a first stage to identify candidate files, then rerank and augment via structural graph traversal (PAR-3). GraphCoder [18] and RepoHyper [27] instantiate this strategy; both outperform pure PAR-2 and pure PAR-3 at the same token budget.

H2: PAR-7 + PAR-3 (agent-guided structural navigation). An agentic system (PAR-7) uses graph-structured indices (PAR-3) as a high-bandwidth navigation oracle rather than raw file reads. LocAgent [2] implements a PAR-7 outer loop that issues Cypher queries against a CodexGraph [20] database, reducing median tool calls from 48 to 21 on SWE-bench (a -56% latency reduction) while retaining resolution quality.

H3: PAR-2 + PAR-6 (efficient large-context retrieval). Retrieve the top-k chunks via PAR-2, then compress the assembled context via PAR-6 before submitting to the LLM. This strategy is applicable when token budget is tight but quality must exceed PAR-6 alone. LongCodeZip [28] can be applied in this post-hoc compression role without modification; the $4\times$ compression rate is largely preserved even on RAG-assembled contexts.

5.6 Open Problems and Research Gaps

The trade-off framework exposes several open problems that the current literature does not adequately address:

(1) **Incremental PAR-3 graph maintenance.** No existing PAR-3 system provides a production-grade incremental graph-update protocol that is robust to API renames, file moves, and dependency removals. HumanEvo [40] quantifies the staleness penalty but does not provide a remedy. A formal change-propagation algorithm for code-context graphs—analogous to incremental compilers in language tooling—is an open engineering problem.

(2) **Precision-preserving compression for PAR-6.** Current compression models [28] are trained to minimise generation loss, which is a poor proxy for task-relevant information retention. A compression objective that explicitly penalises the elision of structurally critical tokens (type annotations, exception signatures, API contracts) is absent from the literature.

(3) **Scalable PAR-7 trajectory control.** Prometheus [26] documents horizon collapse but does not solve it. A principled algorithm for *adaptive trajectory truncation*—terminating navigation when information gain per tool call falls below a threshold— would substantially improve PAR-7 scalability.

(4) **Multi-project and cross-repository representation.** All seven PAR paradigms assume a single-repository context. Microservice architectures [14] and monorepos with loosely coupled sub-packages violate this assumption. The interaction between inter-repository dependency graphs and intra-repository context representation is unexplored.

(5) **Unified evaluation protocol across dimensions.** Benchmark suites including SWE-bench [15], RepoBench [19], and HumanEvo [40] each capture one or two of

the five dimensions (primarily $\mathcal{D}_1$) but none jointly measures all five. A community-standard benchmark that reports latency, update cost, and scalability alongside task quality would substantially accelerate comparative research.

5.7 Summary of RQ3

Answer to RQ3. *No single repository representation paradigm is optimal across all five operational dimensions. PAR-2 (RAG cross-file) provides the best all-round baseline, particularly for latency-constrained interactive tools. PAR-3 (graph-based) maximises semantic precision for stable repositories but is fragile under high commit rates. PAR-6 (compression) is the most token-efficient and scalable paradigm but sacrifices structural precision. PAR-7 (agentic navigation) is the only paradigm that reliably resolves open-ended repository issues, but its latency and scalability limitations restrict its applicability to batch or asynchronous deployment contexts. Practitioners operating near the Pareto frontier of the cost–quality surface should adopt hybrid strategy H1 (PAR-2 + PAR-3) for structured completion tasks and H2 (PAR-7 + PAR-3) for issue-resolution tasks, reserving pure PAR-6 for large-repository summarisation pipelines.*

6 Discussion, Implications, and Limitations

6.1 Implications for Researchers

The formal trade-off framework (§5) reveals that the research community has disproportionately optimised for $\mathcal{D}_1$ (semantic precision) while under-investing in $\mathcal{D}_4$ (update cost) and $\mathcal{D}_5$ (scalability). This imbalance is partly an artefact of benchmark design: SWE-bench [15] and RepoBench [19] are static snapshots that do not penalise stale representations. HumanEvo [40] is a notable exception, and its adoption as a primary benchmark for future PAR-3 evaluations is strongly recommended.

6.2 Implications for Practitioners

System designers should treat the five-dimension framework as a *requirements elicitation* tool: before selecting a paradigm, explicitly specify hard constraints on each dimension (*e.g.* "latency $\leq$ 200 ms", "update cost $\leq$ 5 s", "repository size up to 50K files"). The decision guidelines G1–G5 (§5.4) then yield a unique paradigm recommendation or a hybrid strategy.

6.3 Threats to Validity

Internal validity: the 1–5 dimension scores are based on evidence from primary studies that use heterogeneous experimental setups. We report evidence directly from each study rather than averaging incompatible numbers, but a meta-analytic synthesis would provide stronger quantitative ground.

External validity: the corpus skews toward Python repositories (SWE-bench is Python-only) and may not generalise to statically-typed languages (Java, C++) where PAR-3 tooling is more mature and PAR-2 baselines are weaker.

Construct validity: our five dimensions are not independent; token efficiency ($\mathcal{D}_2$) and construction latency ($\mathcal{D}_3$) are correlated. Future work should apply factor analysis to validate the dimension structure.

7 Related Work

Hou et al.'s SLR [12] is the broadest survey of LLMs for software engineering, covering 395 studies across 9 SE tasks. Our survey complements it by focusing specifically on the *representation* layer for repository-level tasks. Husein et al. [13] survey LLMs for code completion, overlapping our PAR-2 and PAR-6 coverage, but do not consider graph-based, agentic, or memory-augmented paradigms. Tao et al. [29] survey RAG for code generation with a focus on retrieval mechanisms, largely corresponding to our PAR-2 category; they do not provide a multi-dimensional trade-off analysis. Jiang et al. [14] survey code generation broadly and dedicate one section to context handling, but do not propose a formal framework. Zhang et al. [37] cover SE with LLMs at a macro level. None of the above surveys define the five-dimension operational framework of Section 5 or provide practitioner selection guidelines grounded in it.

8 Conclusion

We presented the first systematic literature review focused on *how* LLM-based agents represent code repositories, covering 163 primary studies (2020–2026). We proposed a taxonomy of seven Paradigm of Automated Repository Representation (PAR) categories and characterised them on ten structural dimensions. Our central contribution is a *formal trade-off framework* that evaluates each paradigm on five operational dimensions—Semantic Precision, Token Efficiency, Construction Latency, Update Cost, and Scalability—and translates the resulting matrix into five practitioner-actionable selection guidelines and three hybrid strategies.

The framework reveals that no paradigm dominates all dimensions, that precision and token efficiency are fundamentally anti-correlated, and that scalability remains an underaddressed challenge—particularly for PAR-3 (graph) and PAR-7 (agentic) systems on repositories exceeding 10K files. We identify five open research problems whose resolution would substantially advance the field.

Our replication package, including the full coded corpus and the dimension-scoring rubric, is available at the repository accompanying this submission.

References

[1] Ramakrishna Bairi, Atharv Sonwane, Aditya Kanade, Vageesh D. C, Arun Iyer, Suresh Parthasarathy, Sriram Rajamani, B. Ashok, and Shashank Shet. 2024. CodePlan: Repository-Level Coding using LLMs and Planning. In *Proceedings of FMEC*.

[2] Junkai Chen et al. 2025. SWE-Exp: Experience-Driven Software Issue Resolution. In *arXiv preprint*.

[3] Qinyun Cheng et al. 2024. Dataflow-Guided Retrieval Augmentation for Repository-Level Code Completion. In *Proceedings of ACL*.

[4] Daniela S. Cruzes and Tore Dybå. 2011. Recommended Steps for Thematic Synthesis in Software Engineering. (2011), 275–284.

[5] Shreyas Dhulshette et al. 2025. Hierarchical Repository-Level Code Summarization for Business Applications Using Local LLMs. *arXiv preprint* (2025).

[6] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Guo, Shuai Liu, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*. 1536–1547.

[7] Chun Gu et al. 2025. What to Retrieve for Effective Retrieval-Augmented Code Generation: An Empirical Study and Beyond. In *Proceedings of ISSTA*.

[8] Hang Gu et al. 2025. SECRET: Towards Scalable and Efficient Code Retrieval via Segmented Deep Hashing. *arXiv preprint* (2025).

[9] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. 2022. UniXcoder: Unified Cross-Modal Pre-training for Code Representation. In *Proceedings of ACL*.

[10] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Liu Shujie, Long Zhou, Nan Duan, Daxin Jiang, Ambrosio Blanco, and Ming Zhou. 2021. GraphCodeBERT: Pre-training Code Representations with Data Flow. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

[11] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In *Proceedings of ICLR*.

[12] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large Language Models for Software Engineering: A Systematic Literature Review. *ACM Transactions on Software Engineering and Methodology* 33, 8 (2024), 1–79.

[13] Islem Husein et al. 2025. Large Language Models for Code Completion: A Systematic Literature Review. *arXiv preprint* (2025).

[14] Juyong Jiang et al. 2026. A Survey on Large Language Models for Code Generation. *arXiv preprint* (2026).

[15] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*.

[16] Barbara Kitchenham and Stuart Charters. 2007. Guidelines for Performing Systematic Literature Reviews in Software Engineering. *EBSE Technical Report EBSE-2007-01* (2007).

[17] Tao Liang, Mingwei Liu, et al. 2024. RepoGenix: Dual Context-Aided Repository-Level Code Completion with Language Models. In *Proceedings of ASE*.

[18] Lichen Liu, Mingwei Liu, et al. 2024. GraphCoder: Enhancing Repository-Level Code Completion via Coarse-to-fine Retrieval Based on Code Context Graph. In *Proceedings of ASE*.

[19] Tianyang Liu, Canwen Xu, and Julian McAuley. 2024. RepoBench: Benchmarking Repository-Level Code Auto-Completion Systems. (2024).

[20] Xiangyan Liu et al. 2025. CodexGraph: Bridging Large Language Models and Code Repositories via Code Graph Databases. *arXiv preprint* (2025).

[21] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, Ge Li, Lidong Zhou, Linjun Shou, Long Zhou, Michele Tufano, Ming Gong, Ming Zhou, Nan Duan, Neel Sundaresan, Shao Kun Deng, Shengyu Fu, and Shujie Liu. 2022. ReACC: A Retrieval-Augmented Code Completion Framework. In *Proceedings of ACL*. 6227–6240.

[22] Hao Luo et al. 2026. Closing the Loop: Universal Repository Representation with RPG-Encoder. *arXiv preprint* (2026).

[23] Yingwei Ma et al. 2025. Alibaba LingmaAgent: Improving Automated Issue Resolution via Comprehensive Repository Exploration. *arXiv preprint* (2025).

[24] Siru Ouyang et al. 2025. RepoGraph: Enhancing AI Software Engineering with Repository-level Code Graph. *arXiv preprint* (2025).

[25] Matthew J. Page, Joanne E. McKenzie, Patrick M. Bossuyt, Isabelle Boutron, Tammy C. Hoffmann, Cynthia D. Mulrow, Larissa Shamseer, Jennifer M. Tetzlaff, Elie A. Akl, Sue E. Brennan, et al. 2021. PRISMA 2020 Explanation and Elaboration: Updated Guidance and Exemplars for Reporting Systematic Reviews. *BMJ* 372 (2021), n160.

[26] Yu Pan et al. 2026. Prometheus: Towards Long-Horizon Codebase Navigation for Repository-Level Problem Solving. *arXiv preprint* (2026).

[27] Ha Minh Phan et al. 2025. RepoHyper: Search-Expand-Refine on Semantic Graphs for Repository-Level Code Completion. In *Proceedings of EMNLP*.

[28] Yuqi Shi et al. 2025. LongCodeZip: Compress Long Context for Code Language Models. *arXiv preprint* (2025).

[29] Chao Tao et al. 2026. Retrieval-Augmented Code Generation: A Survey with Focus on Repository-Level Approaches. *arXiv preprint* (2026).

[30] Shuai Wang et al. 2026. CodeMEM: AST-Guided Adaptive Memory for Repository-Level Iterative Code Generation. *arXiv preprint* (2026).

[31] Xingyao Wang et al. 2025. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. *arXiv preprint* (2025).

[32] Yuetian Wang et al. 2026. SWE-Pruner: Self-Adaptive Context Pruning for Coding Agents. In *arXiv preprint*.

[33] Claes Wohlin. 2014. Guidelines for Snowballing in Systematic Literature Studies and a Replication in Software Engineering. In *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering (EASE)*. ACM, 1–10.

[34] Di Wu, Wasi Uddin Ahmad, Dejiao Zhang, Murali Krishna Ramanathan, and Xiaofei Ma. 2024. Repoformer: Selective Retrieval for Repository-Level Code Completion. In *Proceedings of ICML*.

[35] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[36] Fengji Zhang, Bei Chen, Yue Zhang, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. 2023. RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation. In *Proceedings of EMNLP*. 2471–2484.

[37] Quanjun Zhang et al. 2026. A Survey on Large Language Models for Software Engineering. *arXiv preprint* (2026).

[38] Shu Zhang et al. 2025. Hierarchical Context Pruning: Optimizing Real-World Code Completion with Repository-Level Pretrained Models. *arXiv preprint* (2025).

[39] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. AutoCodeRover: Autonomous Program Improvement. In *Proceedings of ISSTA*.

[40] Siwei Zheng et al. 2025. HumanEvo: An Evolution-aware Benchmark for More Realistic Evaluation of Repository-Level Code Generation. *arXiv preprint arXiv:2503.XXXXX* (2025).